

Anexo 16 — Logica del cliente web (JavaScript)

Archivo: Scripts/web_remote/static/control.js

Para que sirvio:

Captura el arrastre del objetivo en el canvas, dibuja el pad, limita el objetivo al rango de entrenamiento, envia los comandos y muestra la telemetria recibida.

```
1 | // Control remoto del target del DUM Pit Droid.
2 | // Canvas representa el plano (X lateral, Z vertical) en world frame.
3 | // Cliente envia {x, y, z} cada vez que el usuario mueve el target.
4 | // Servidor envia telemetria {theta_deg, focus, fps}.
5 |
6 | const PAD = document.getElementById("pad");
7 | const CTX = PAD.getContext("2d");
8 | const STATUS = document.getElementById("status");
9 | const DIST_SLIDER = document.getElementById("dist-slider");
10 | const DIST_VAL = document.getElementById("dist-val");
11 | const THETA_VAL = document.getElementById("theta-val");
12 | const FOCUS_VAL = document.getElementById("focus-val");
13 | const FOCUS_CELL = document.getElementById("focus-cell");
14 | const FPS_VAL = document.getElementById("fps-val");
15 | const STATE_VAL = document.getElementById("state-val");
16 | const GRAB_BTN = document.getElementById("grab-btn");
17 | const WAVE_BTN = document.getElementById("wave-btn");
18 | const SKIN_REALISTA = document.getElementById("skin-realista");
19 | const SKIN_IMPERIO = document.getElementById("skin-imperio");
20 |
21 | // Rango de coordenadas WORLD que cubre el pad. Calibrado al modelo real:
22 | // posicion del lente en pose neutra (medida con mujoco.site_xpos['lens_center']).
23 | // - X world: lateral. Pad horizontal -> X world.
24 | // - Z world: vertical. Pad vertical -> Z world (invertido para que arriba=+Z).
25 | // - Y world: distancia frontal/trasera. Controlada por el slider con signo.
26 | // El robot mira hacia -Y world en pose neutra. Distancia positiva = al frente (-Y world);
27 | // negativa = detras (+Y world). El slider va de -1 (atras) a +1 (adelante).
28 | const LENS_POS_WORLD = { x: 0.115, y: 0.139, z: 0.503 };
29 | const PAD_RANGE_X = 1.2; // metros (ancho del pad en world X). +/- 0.6 a cada lado del lente.
30 | const PAD_RANGE_Z = 1.0; // metros (alto del pad en world Z). +/- 0.5 (arriba/abajo).
31 | // v14c training distancia 0.3..1.0m. Limite el slider para no salirse del rango entrenado.
32 | const DIST_MIN = -0.65; // backward dentro del cono ±150° azimuth y dist <= 0.7m
33 | const DIST_MAX = 0.95; // frente, max 0.95m (training: 1.0m max, dejo margen)
34 | // Clamp final del target para que la distancia 3D al lens este en [TRAIN_DIST_MIN, TRAIN_DIST_MAX]
35 | const TRAIN_DIST_MIN = 0.30;
36 | const TRAIN_DIST_MAX = 1.00;
37 |
38 | // Estado del target en world coords
39 | let target = {
40 |   x: LENS_POS_WORLD.x,
41 |   y: LENS_POS_WORLD.y - 0.6, // arranca 0.6m al frente
42 |   z: LENS_POS_WORLD.z,
43 | };
44 | let signedDist = 0.6; // -0.8 .. +1.0
45 |
46 | let dragging = false;
47 |
48 | // --- canvas drawing ---
49 | function drawPad() {
50 |   const w = PAD.width, h = PAD.height;
51 |   // bg
52 |   CTX.fillStyle = "#2a2a30";
53 |   CTX.fillRect(0, 0, w, h);
54 |
55 |   // grid
56 |   CTX.strokeStyle = "#3a3a45";
57 |   CTX.lineWidth = 1;
58 |   for (let i = 1; i < 8; i++) {
59 |     const x = (w * i) / 8;
60 |     const y = (h * i) / 8;
61 |     CTX.beginPath();
62 |     CTX.moveTo(x, 0); CTX.lineTo(x, h);
63 |     CTX.moveTo(0, y); CTX.lineTo(w, y);
64 |     CTX.stroke();
65 |   }
66 |
67 |   // ejes centrales
68 |   CTX.strokeStyle = "#555";
69 |   CTX.lineWidth = 1.5;
70 |   CTX.beginPath();
71 |   CTX.moveTo(w / 2, 0); CTX.lineTo(w / 2, h);
72 |   CTX.moveTo(0, h / 2); CTX.lineTo(w, h / 2);
73 |   CTX.stroke();
74 | }
```

```

75 | // etiquetas
76 | CTX.fillStyle = "#999";
77 | CTX.font = "12px system-ui";
78 | CTX.fillText("← izquierda", 6, h / 2 - 6);
79 | CTX.fillText("derecha →", w - 70, h / 2 - 6);
80 | CTX.fillText("↑ arriba", w / 2 + 6, 14);
81 | CTX.fillText("↓ abajo", w / 2 + 6, h - 8);
82 |
83 | // target
84 | const px = ((target.x - LENS_POS_WORLD.x) / PAD_RANGE_X + 0.5) * w;
85 | const py = ((LENS_POS_WORLD.z - target.z) / PAD_RANGE_Z + 0.5) * h; // Z arriba = py chico
86 | CTX.beginPath();
87 | CTX.arc(px, py, 18, 0, Math.PI * 2);
88 | CTX.fillStyle = "#ff4747";
89 | CTX.fill();
90 | CTX.strokeStyle = "#fff";
91 | CTX.lineWidth = 2;
92 | CTX.stroke();
93 | }
94 |
95 | // --- pad -> world ---
96 | function padToWorld(px, py) {
97 |   const w = PAD.width, h = PAD.height;
98 |   const x = LENS_POS_WORLD.x + (px / w - 0.5) * PAD_RANGE_X;
99 |   const z = LENS_POS_WORLD.z - (py / h - 0.5) * PAD_RANGE_Z;
100 |   return { x, z };
101 | }
102 |
103 | // --- WebSocket ---
104 | let ws;
105 | function connect() {
106 |   const proto = location.protocol === "https:" ? "wss:" : "ws:";
107 |   ws = new WebSocket(`${proto}/${location.host}/ws`);
108 |   ws.onopen = () => {
109 |     STATUS.textContent = "conectado";
110 |     STATUS.className = "ok";
111 |     sendTarget(); // envia el target inicial
112 |   };
113 |   ws.onclose = () => {
114 |     STATUS.textContent = "desconectado - reintentando...";
115 |     STATUS.className = "err";
116 |     setTimeout(connect, 1500);
117 |   };
118 |   ws.onerror = () => {
119 |     STATUS.textContent = "error";
120 |     STATUS.className = "err";
121 |   };
122 |   ws.onmessage = (e) => {
123 |     try {
124 |       const t = JSON.parse(e.data);
125 |       if (typeof t.theta_deg === "number") {
126 |         THETA_VAL.textContent = t.theta_deg.toFixed(1) + " °";
127 |       }
128 |       if (typeof t.focus === "boolean") {
129 |         FOCUS_VAL.textContent = t.focus ? "YES" : "no";
130 |         FOCUS_VAL.className = "val " + (t.focus ? "YES" : "no");
131 |       }
132 |       if (typeof t.fps === "number") {
133 |         FPS_VAL.textContent = t.fps.toFixed(0);
134 |       }
135 |       if (typeof t.state === "string" && STATE_VAL) {
136 |         STATE_VAL.textContent = t.state;
137 |         STATE_VAL.style.color = (t.state === "IDLE") ? "var(--mute)" : "var(--ok)";
138 |         // Actualizar habilitacion de botones segun el SM real (no por timeout)
139 |         if (t.state !== currentSMState) {
140 |           currentSMState = t.state;
141 |           updateButtonStateFromSM();
142 |         }
143 |       }
144 |     } catch (_) {}
145 |   };
146 | }
147 |
148 | // Estado actual del SM segun telemetria (se actualiza desde ws.onmessage)
149 | let currentSMState = "IDLE";
150 |
151 | function updateButtonStateFromSM() {
152 |   // Los botones de accion solo se habilitan en IDLE
153 |   const inIdle = (currentSMState === "IDLE");
154 |   if (GRAB_BTN) {
155 |     GRAB_BTN.disabled = !inIdle;
156 |     if (inIdle) GRAB_BTN.textContent = "AGARRAR BOLA";
157 |     else GRAB_BTN.textContent = "AGARRAR (esperá: " + currentSMState + ")";
158 |   }
159 |   if (WAVE_BTN) {

```

```

160 |     WAVE_BTN.disabled = !inIdle;
161 |     if (inIdle) WAVE_BTN.textContent = "SALUDAR (procedural 6s)";
162 |     else WAVE_BTN.textContent = "SALUDAR (esperá: " + currentSMSState + ")";
163 | }
164 | }
165 |
166 | if (GRAB_BTN) {
167 |     GRAB_BTN.addEventListener("click", async () => {
168 |         if (currentSMSState !== "IDLE") return;
169 |         GRAB_BTN.disabled = true;
170 |         GRAB_BTN.textContent = "...";
171 |         try {
172 |             await fetch("/grab_yellow", { method: "POST" });
173 |         } catch (e) {
174 |             GRAB_BTN.textContent = "ERROR de red";
175 |         }
176 |         // No timeout: la habilitacion vuelve cuando la telemetria reporta IDLE
177 |     });
178 | }
179 |
180 | if (WAVE_BTN) {
181 |     WAVE_BTN.addEventListener("click", async () => {
182 |         if (currentSMSState !== "IDLE") return;
183 |         WAVE_BTN.disabled = true;
184 |         WAVE_BTN.textContent = "...";
185 |         try {
186 |             await fetch("/wave", { method: "POST" });
187 |         } catch (e) {
188 |             WAVE_BTN.textContent = "ERROR de red";
189 |         }
190 |     });
191 | }
192 |
193 | // Botones de apariencia (skin) – funcionan en cualquier estado, son cosméticos
194 | async function setSkin(name) {
195 |     try { await fetch("/skin/" + name, { method: "POST" }); } catch (e) {}
196 |     // Marcar el activo
197 |     if (SKIN_REALISTA) SKIN_REALISTA.classList.toggle("skin-active", name === "realista");
198 |     if (SKIN_IMPERIO) SKIN_IMPERIO.classList.toggle("skin-active", name === "imperio");
199 | }
200 | if (SKIN_REALISTA) SKIN_REALISTA.addEventListener("click", () => setSkin("realista"));
201 | if (SKIN_IMPERIO) SKIN_IMPERIO.addEventListener("click", () => setSkin("imperio"));
202 |
203 | // D-pad de cámara: izq/der giran, arriba/abajo cambian altura (cosmético)
204 | async function moveCam(dir) {
205 |     try { await fetch("/cam/" + dir, { method: "POST" }); } catch (e) {}
206 | }
207 | ["up", "down", "left", "right"].forEach((dir) => {
208 |     const btn = document.getElementById("cam-" + dir);
209 |     if (btn) btn.addEventListener("click", () => moveCam(dir));
210 | });
211 |
212 | function clampTargetToTrainingDist(t) {
213 |     // Asegura que la distancia 3D del target al lens este en [TRAIN_DIST_MIN, TRAIN_DIST_MAX].
214 |     // Si esta fuera, se mueve a lo largo del rayo lens->target hasta caer en rango.
215 |     const dx = t.x - LENS_POS_WORLD.x;
216 |     const dy = t.y - LENS_POS_WORLD.y;
217 |     const dz = t.z - LENS_POS_WORLD.z;
218 |     const d = Math.sqrt(dx*dx + dy*dy + dz*dz);
219 |     if (d < 1e-6) return t; // protege contra div/0
220 |     let factor = 1.0;
221 |     if (d > TRAIN_DIST_MAX) factor = TRAIN_DIST_MAX / d;
222 |     else if (d < TRAIN_DIST_MIN) factor = TRAIN_DIST_MIN / d;
223 |     return {
224 |         x: LENS_POS_WORLD.x + dx * factor,
225 |         y: LENS_POS_WORLD.y + dy * factor,
226 |         z: LENS_POS_WORLD.z + dz * factor,
227 |     };
228 | }
229 |
230 | function sendTarget() {
231 |     if (ws && ws.readyState === WebSocket.OPEN) {
232 |         const clamped = clampTargetToTrainingDist(target);
233 |         ws.send(JSON.stringify(clamped));
234 |     }
235 | }
236 |
237 | // --- input events ---
238 | function updateFromPointer(ev) {
239 |     const rect = PAD.getBoundingClientRect();
240 |     const ptr = ev.touches ? ev.touches[0] : ev;
241 |     const px = ((ptr.clientX - rect.left) / rect.width) * PAD.width;
242 |     const py = ((ptr.clientY - rect.top) / rect.height) * PAD.height;
243 |     const w = padToWorld(px, py);
244 |     target.x = w.x;

```

```

245 |   target.z = w.z;
246 |   drawPad();
247 |   sendTarget();
248 | }
249 |
250 |   PAD.addEventListener("pointerdown", (e) => { dragging = true; PAD.setPointerCapture(e.pointerId);
updateFromPointer(e); });
251 | PAD.addEventListener("pointermove", (e) => { if (dragging) updateFromPointer(e); });
252 | PAD.addEventListener("pointerup", (e) => { dragging = false; });
253 | PAD.addEventListener("pointercancel", (e) => { dragging = false; });
254 |
255 | function applyDist() {
256 |   // d signed: positivo = adelante (-Y world); negativo = atras (+Y world).
257 |   signedDist = parseFloat(DIST_SLIDER.value) / 100;
258 |   if (signedDist >= 0) {
259 |     DIST_VAL.textContent = signedDist.toFixed(2) + " m frente";
260 |   } else {
261 |     DIST_VAL.textContent = Math.abs(signedDist).toFixed(2) + " m atras";
262 |   }
263 |   // target.y = lente.y - dist_signed (frente = -Y, atras = +Y)
264 |   target.y = LENS_POS_WORLD.y - signedDist;
265 |   sendTarget();
266 | }
267 | DIST_SLIDER.addEventListener("input", applyDist);
268 |
269 | // Init
270 | drawPad();
271 | connect();
272 |

```